

MCP Target Routing

One gateway, many machines: teaching a single MCP deployment to reach whichever Docker engine, Ignition gateway, or database a request names — instead of wiring each server to one target at deploy time.

scope: backend/mcp-gateway, docker-mcp, ignition-mcp, postgres-mcp 2026-08-15

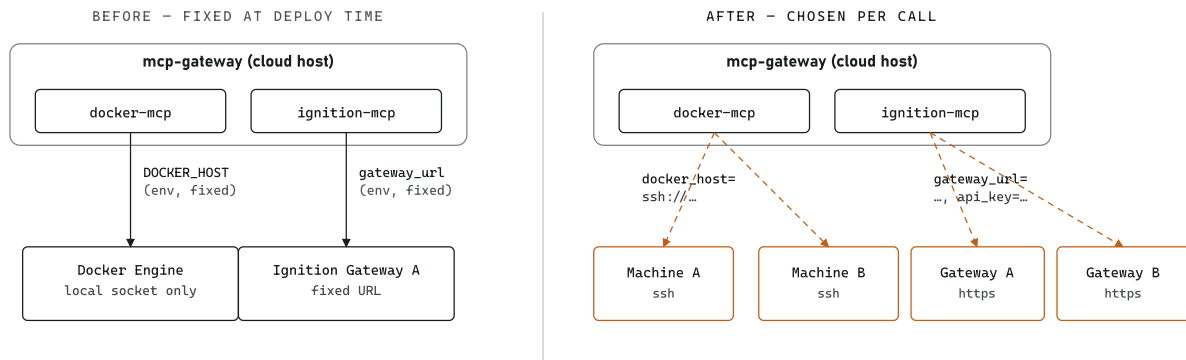
01 The goal

Right now, running an MCP tool means running it *somewhere specific* — the target it talks to is baked into how and where it's deployed. The goal is to invert that: host the MCP servers once, on a single cloud Linux box, and let every tool call name its own target through a URL and a credential.

"What containers are running on machine X" should resolve against whichever engine X names — the model fills in X and a credential as part of the call, and the CLI never needs to know or care where the MCP server process itself happens to live.

02 Before → after

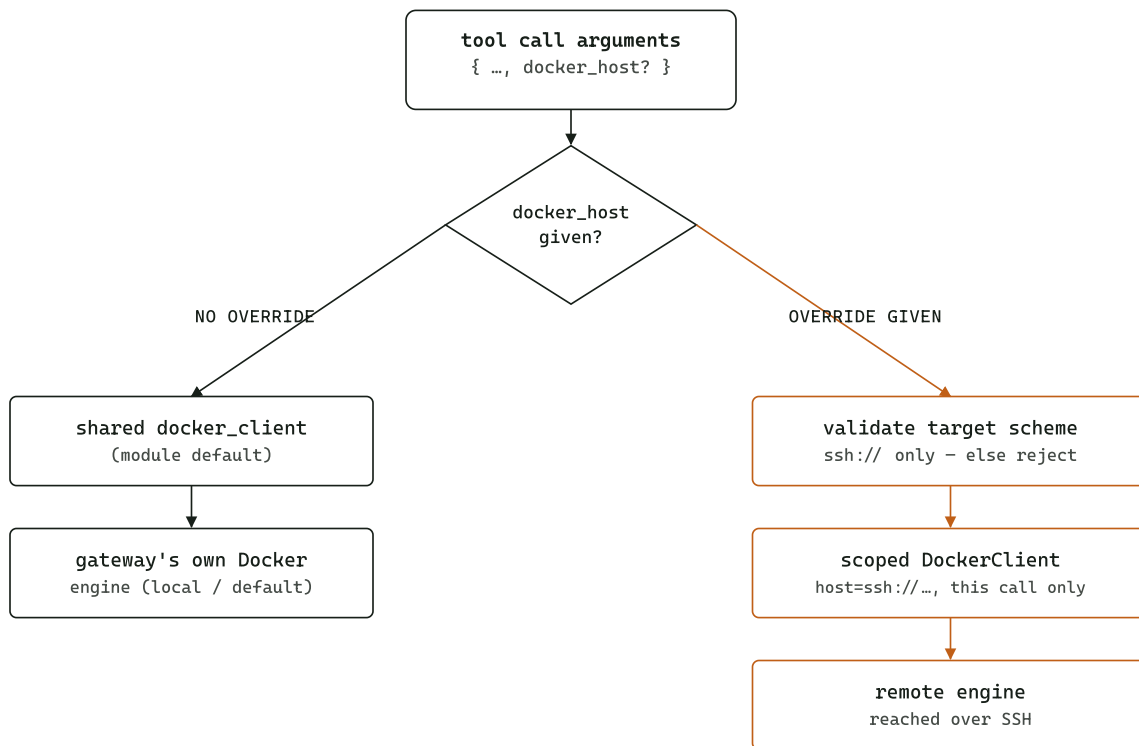
ignition-mcp already proved the pattern: `gateway_url` and `api_key` travel as call parameters, not startup config. docker-mcp didn't have it — every tool call reused one `DockerClient` built when the process started, pointed at whatever engine that process could see. Centralizing both servers behind `mcp-gateway` didn't change that on its own; it just put two still single-target servers behind one door.



Solid grey arrows mark the old wiring, fixed when the server was deployed. Dashed amber arrows mark the new per-call routing — the target is chosen by a parameter in that tool call's own arguments, so one deployment now reaches several machines.

03 How a call picks its target

Both servers use the same mechanism: a tool's arguments carry an optional override, and a small scoping function decides — per call — whether to reuse the shared default client or build a disposable one pointed elsewhere. For docker-mcp that override is `docker_host`, and it accepts only the `ssh://` scheme, on purpose.



The same encoding continues: grey follows the no-override path to the gateway's own engine; amber follows the `ssh://` override path to a scoped, disposable client built just for that call.

TCP+TLS access to a remote Docker daemon needs a client certificate, key, and CA file — none of which have a safe way to travel as a plaintext tool argument; a model constructing the call would see them, and they'd sit in the conversation transcript. An SSH target needs only a host string. The key material stays on disk on whichever machine runs `docker-mcp`, never inside a tool call.

04 What changed in `docker-mcp`

The module-level client became a function that branches on the override, in `handlers.py`:

BEFORE

```

docker_client = DockerClient()

async def handle_list_containers(arguments):
    ...
    containers = await asyncio.to_thread(

```

AFTER

```

docker_client = DockerClient() # default,

def get_docker_client(docker_host=None):
    if not docker_host:
        return docker_client

```

```

        docker_client.container.list,
        all=True,
    )

```

```

        if not docker_host.startswith("ssh://"):
            raise ValueError(
                f"docker_host must use ssh:// "
                f"(got {docker_host!r})"
            )
        return DockerClient(host=docker_host)

    async def handle_list_containers(arguments):
        ...
        client = get_docker_client(
            (arguments or {}).get("docker_host")
        )
        containers = await asyncio.to_thread(
            client.container.list,
            all=True,
        )

```

Two more spots needed the same override. `deploy-compose` doesn't go through `python_on_whales` at all — it shells out to the `docker compose` CLI directly, so `docker_executor.py` now injects the target as a flag on that command instead:

```
docker -H ssh://user@remote-host compose -f project-docker-compose.yml -p project up -d
```

And `server.py` gained a documented `docker_host` property on all four tool schemas, so the override is discoverable by the model calling the tool, not just silently accepted.

05 Status across servers

SERVER	PER-CALL TARGET OVERRIDE	PARAMETER(S)	TRANSPORT
ignition-mcp	Yes — reference implementation	gateway_url, api_key	HTTPS
docker-mcp	Yes — added this change	docker_host	SSH
postgres-mcp	Not yet	—	—

06 Open items

- ▶ Port the same override pattern to postgres-mcp — a connection string or named profile per call.
- ▶ Actually deploy mcp-gateway to the cloud Linux host; everything above still runs locally.
- ▶ Put an SSH client and the private key docker-mcp needs onto that host.
- ▶ Revisit raw override strings vs. named profiles once this leaves a single-operator setup — profiles keep credentials out of the model's context entirely.

[backend/mcp-gateway](#) · [backend/docker-mcp](#) · [backend/ignition/ignition-mcp](#)